

ch-09 : Main Memory

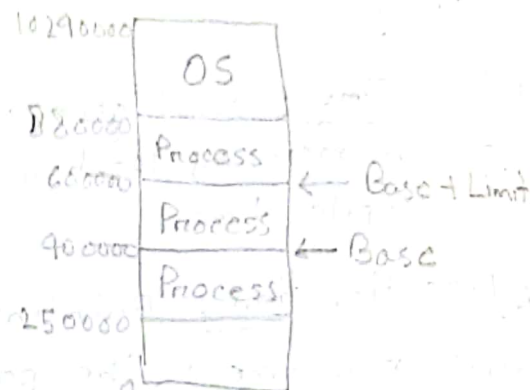
Protection: Need to ensure that a process can only access those addresses in its address space.

The base register holds the smallest legal physical memory address; the limit register specifies the size of the range.

→ We can provide this protection by using a pair of base and limit registers define the logical address space of a process.

→ CPU must check every time memory access generated in user mode to be sure it is between base and limit for that user.

The instructions to loading the base and limit registers are privileged.



A base and limit register define a logical address space.

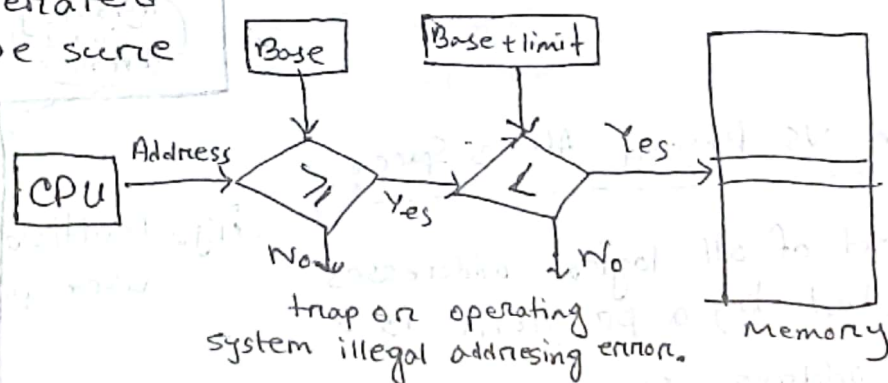


Fig: Hardware address protection with base and limit registers.

Address Binding:

classically, the binding of instructions and data to memory addresses can be done at three different stages:

- Compile Time: If memory location known a priori, absolute code can be generated; must recompile code if starting location change.
- Load Time: Must generate relocatable code if memory location is not known at compile time.
- Execution time: Binding delay until run time if the process can be moved during its execution from one memory segment to another.

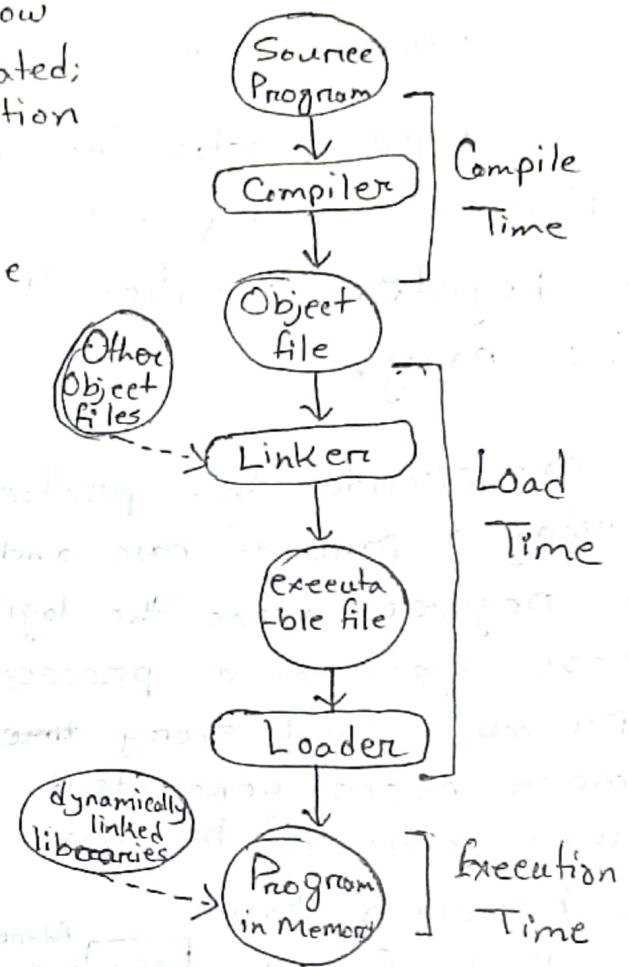


Fig: Multistage processing of a user program.

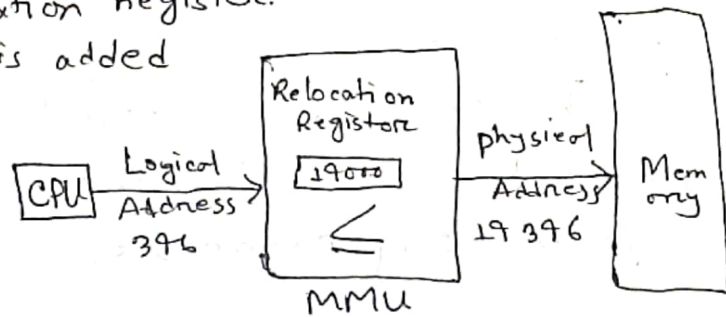
Logical vs Physical Address Space:

- ↳ The set of all logical addresses generated by a program is a logical address space.
- ↳ The set of all physical addresses by a program corresponding to these logical addresses is a physical address space.
- ↳ Logical or virtual address generated by the CPU.
- ↳ Physical address seen by the memory unit.

The run-time mapping from virtual to physical addresses is done by a hardware device called the Memory Management Unit (MMU).

MMU

- The base register now called relocation register.
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory.



⇒ The user program deals with logical addresses; it never sees the real physical address.

Dynamically Loading: The size of a process has thus been limited to the size of physical memory. To obtain better memory-space utilization, we can use dynamically loading.

- ① The entire program does not need to be in memory to execute.
- ② Routing is not loaded until it is called.
- ③ Better memory-space utilization; unused routing is never loaded.

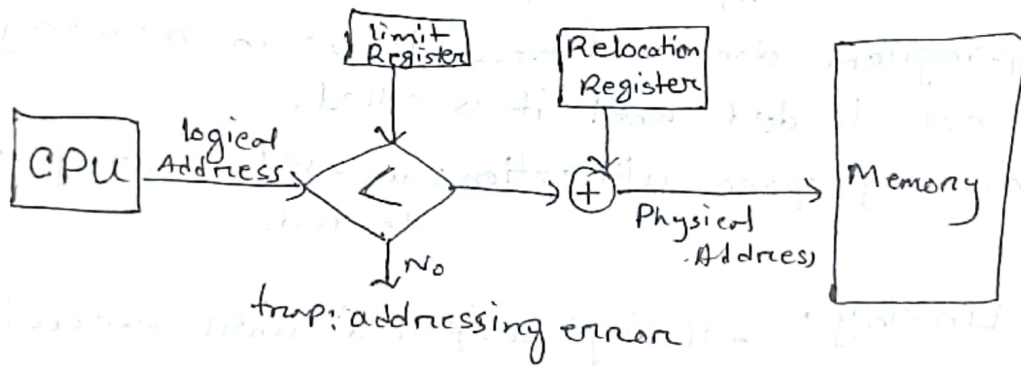
Dynamic Linking: - linking postponed until execution time

Contiguous Memory Allocation:

The memory is usually divided into two partitions, one for the OS and one for the user processes. We can place the OS in either low memory addresses or high memory addresses.

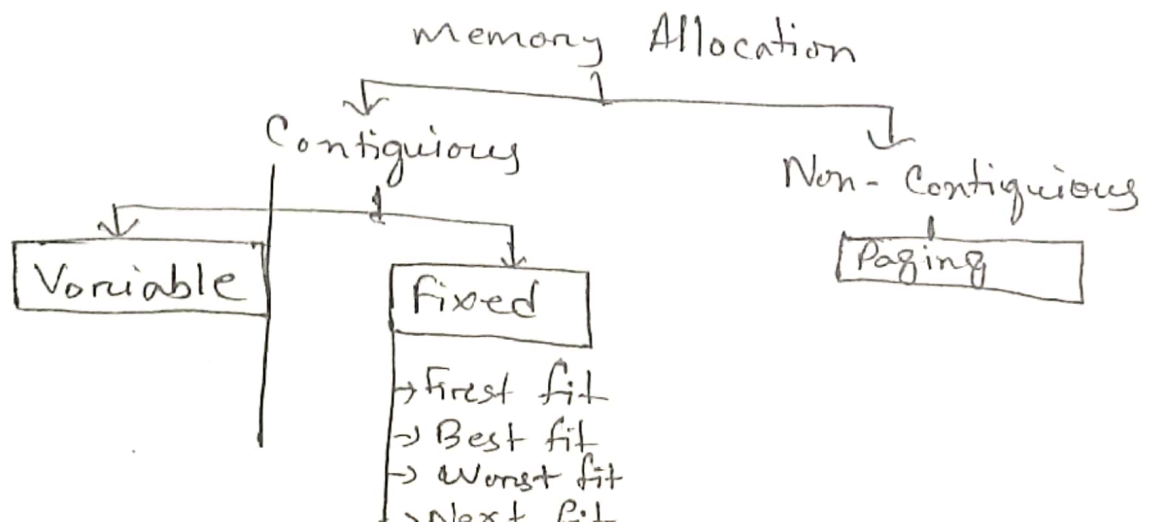
In contiguous memory allocation, each process is contained in a single section of memory that is contiguous to the section containing the next process.

⇒ Hardware support for Relocation and limit Registers:



Memory Allocation:

Initially all memory is available for user processes and is considered one large block of available memory, a hole.



When a process arrives and needs memory, the system searches the set ~~for~~ a hole that is large enough for this process. If the hole is too large, it is split into two parts. One part is allocated to the arriving process; the other is returned to the set of holes. When a process terminates, it releases its block of memory, which is then placed back in the set of holes.

First fit: Allocate ~~first~~ the first hole that is big enough. searching ~~start~~ at the begining of the set of holes.

Next fit: Allocate the first hole that is big enough, searching start at the location where the previous first-fit search ended.

Best fit: Allocate the smallest hole that is big enough. We must search the entire list, unless the list is ordered by size. This strategy produce the smallest leftover hole.

Worst fit: Allocate the biggest hole. Again, we must search the entire list, unless it is sorted by size.

$$P_1 = 20$$

$$P_2 = 12$$

$$P_3 = 15$$

$$P_4 = 10$$

$$P_5 = 5$$

	15
First fit	25
Best fit + next fit	20
	9
	8
	7
Worst fit	30

For P_1 process.

Variable Partition:

Degree of multiprogramming limited by number of partitions

Variable-partition sizes for efficiency.

Whole block of available memory.

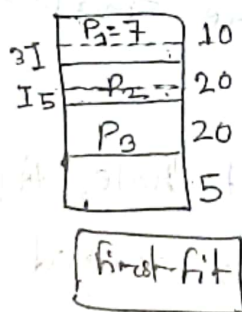
Operating system maintains information about:

- Ⓐ allocated partitions
- Ⓑ free partitions

Fragmentation: ① External ② Internal

⇒ External fragmentation :- total memory space exists to satisfy a request, but it is not contiguous. Both the first-fit and best-fit strategies for memory allocation suffer from external fragmentation.

$P_1 = 7$
 $P_2 = 15$
 $P_3 = 20$
 $P_4 = 20$

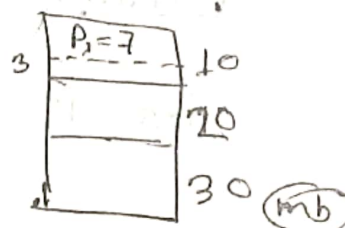


More total available
 $5 + 5 + 3 = 13$ but not contiguous that's why we can not allocate P_4 process

⇒ ② Internal fragmentation - allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used.

here internal frag 3 mb

$P_1 = 7 \text{ mb}$



Paging: A memory management scheme that permits a process's physical address space to be non-contiguous.

⇒ Process is allocated physical memory whenever the latter is available

- Avoids external fragmentation.
- Avoids problem of varying sized memory chunks.

Method

Breaking physical memory into fixed-sized blocks called frame.

$$\text{Frame} = \frac{\text{Physical Ad.}}{\text{block size}}$$

~~Breaking~~

$$\text{Nu. of Page} = \frac{\text{Logical Ad.}}{\text{Page-size}}$$

$$\text{offset} = \text{Logical A.} \% \text{ P-size}$$

→ Breaking logical memory into blocks of the same size called pages.

Every address generated by the CPU is divided into two parts: a page number (p) and page offset (d):

$$\text{Physical Address} = (\text{Frame No} * \text{Page-size}) + \text{Offset}$$

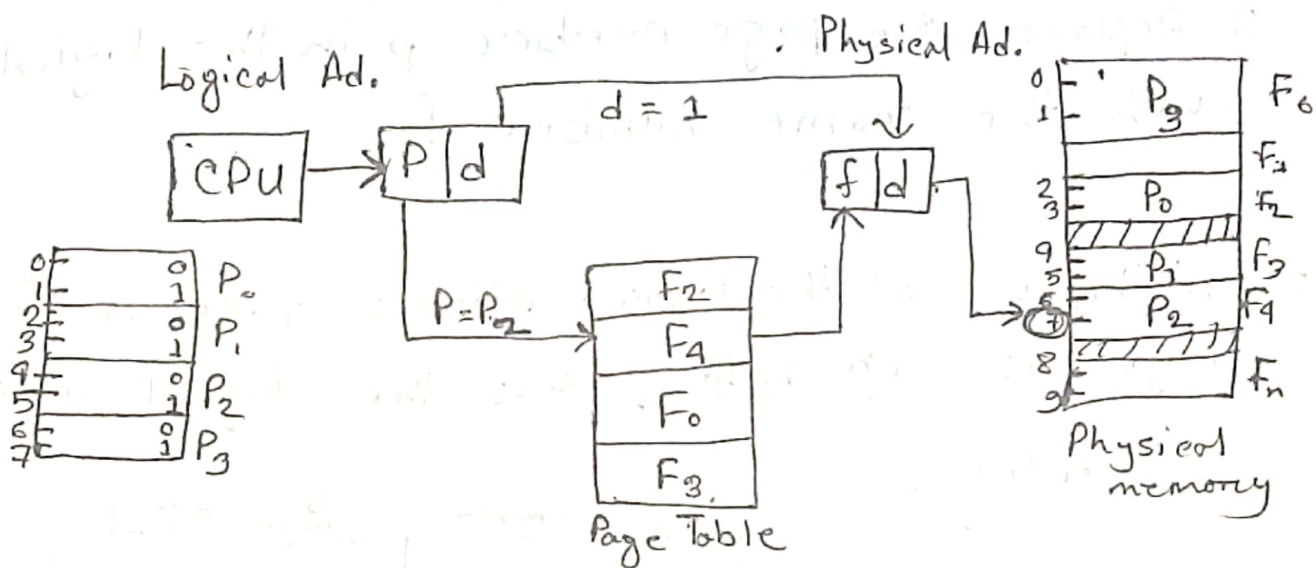


fig: Paging hardware

The page number is used as an index into a per-process page table.

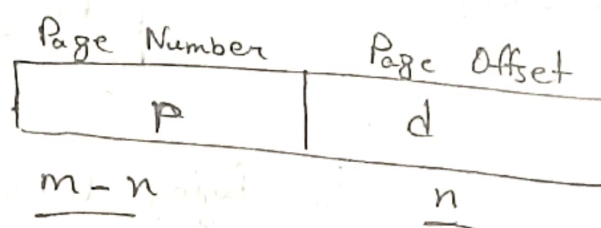
Here,

- Keep track of all free frames
- To run a program of size N pages, need to find N free frames and load program.
- Set up a page table to translate logical to physical addresses.
- Backing store likewise split into page.
- Still have internal fragmentation.

The steps taken by the MMU to translate a logical address generated by the CPU to a physical address:

1. Extract the page number p and use it as an index into the page table.
2. Extract the corresponding frame number f from the page table.
3. Replace the page number p in the logical address with the frame number f .

⇒ If the size of the logical address space is 2^m and a page size 2^n bytes, Thus the logical address is as follow:



Ex: Page size = 2098 bytes; Process size = 72766 bytes

So, need, 35 pages + 1086 bytes

□ Internal fragmentation = $2098 - 1086 = 962$ bytes

□ In worst case, a process need n pages plus 1 byte. It would be allocated $n+1$ frames, resulting in internal fragmentation of almost an entire frame.

here, worst case fragmentation = 1 frame - 1 byte
so on average fragmentation = $\frac{1}{2}$ frame size.

□ So small frame size desirable?

If process size is independent of page size, we expect internal fragmentation to average one-half page per process. This consideration suggest that small page size one desirable.